

Respuesta de las preguntas de la sumisión

David Alejandro Castaño Moreno

Brayan Grisales Perez

Solución de las preguntas del código de Haskell

1. Describe in detail the process you carried out in order to solve the problem stated, indicating the decisions taken in each part of the practice.

Para resolver el problema, primero se dividió la solución en diferentes funciones, de manera que cada una tuviera una responsabilidad específica. En primer lugar, se desarrolló la función `divisoresPropios`, cuya finalidad es encontrar los divisores propios de un número. Para esto se utilizó una lista por comprensión que recorre los números desde 1 hasta el mismo número ingresado y verifica cuáles lo dividen exactamente mediante el operador `mod`.

Después se creó la función `sumaAlicuota`, que recibe un número y calcula la suma de todos sus divisores propios. Para esto se reutilizó la función `divisoresPropios` y se aplicó la función `sum`, evitando repetir el proceso de búsqueda de divisores.

Posteriormente, se implementó la función `categoriaNicomaco`. Esta función utiliza la suma alícuota obtenida anteriormente y la compara con el número original. Dependiendo del resultado de la comparación, asigna una de las tres categorías establecidas: "Administrative" cuando la suma es mayor que el número, "Engineering" cuando son iguales y "Humanities" cuando la suma es menor.

Finalmente, se desarrolló la función `procesarCodigo`, encargada de integrar todos los procesos anteriores. Primero se verifica que el código tenga exactamente ocho caracteres y que el período de admisión pertenezca a uno de los períodos permitidos. Si alguna de estas condiciones no se cumple, se muestra un mensaje de error. Si el código es válido, se extraen sus diferentes partes utilizando funciones como `take` y `drop`: los tres primeros caracteres corresponden al período, los siguientes dos a la categoría y los últimos tres al número consecutivo. Luego en la función `formaPeriodoBuena` se pone el período de admisión de la forma en la que se pide, se calcula la categoría utilizando la función `categoriaNicomaco` y se determina

si el código es par o impar. Finalmente, todos estos resultados se unen mediante unwords para producir la respuesta final.

Como comprobación de la solución, en la función main se utilizaron diferentes códigos de ejemplo para verificar que el programa generara los resultados esperados.

Sobre las decisiones al momento de hacer el código creo que la más fundamental fue la de dividir el problema en funciones pequeñas y reutilizables, ya que esto permite mantener una estructura más clara y facilita la comprensión y comprobación del programa, simplificando así mucho el código

2. Describe a way in which the currying process is manifested in the proposed solution.

El currying se manifiesta en funciones como divisoresPropios, sumaAlicuota, categoriaNicomaco y procesarCodigo, ya que cada una recibe un único argumento. Por ejemplo, divisoresPropios :: Int -> [Int] recibe un valor de tipo Int y genera como resultado una lista de enteros.

Hay que aclarar que en la solución no se utiliza explícitamente una función con varios argumentos para realizar una aplicación parcial, por lo que el currying se evidencia principalmente en la definición de las funciones mediante un solo parámetro.

3. Indicate, for each function developed, whether you identify it as polymorphic, as higher-order, or whether you do not consider it classified under either or under both criteria; justify each determination

La función divisoresPropios no es polimórfica ni de orden superior, ya que trabaja específicamente con valores de tipo Int y devuelve una lista de Int. Además, no recibe ni devuelve funciones.

La función sumaAlicuota tampoco es polimórfica ni de orden superior, porque recibe un Int y devuelve un Int. Aunque utiliza la función sum, no recibe una función como argumento ni devuelve una función.

La función categoriaNicomaco no es polimórfica ni de orden superior, debido a que recibe un Int y devuelve un String. Su funcionamiento se basa en comparar valores y no utiliza funciones como argumentos o resultados.

La función `procesarCodigo` tampoco es polimórfica ni de orden superior, ya que recibe un `String` y devuelve un `String`. Aunque utiliza varias funciones auxiliares y funciones propias de Haskell, no recibe ninguna función como argumento ni devuelve una función.

La función auxiliar `formaPeriodoBuena` no es polimórfica ni de orden superior, porque recibe un `String` y devuelve un `String`, sin trabajar con funciones como argumentos.

La función auxiliar `paridad` tampoco es polimórfica ni de orden superior, ya que recibe un `String` y devuelve un `String`. Aunque utiliza `even`, no recibe una función como parámetro ni devuelve una función.

Finalmente, `main` no se considera polimórfica ni de orden superior, ya que corresponde a una acción de tipo `IO ()` y no recibe ni devuelve funciones.

En conclusión, ninguna de las funciones desarrolladas directamente en la solución se clasifica como polimórfica o de orden superior.

Aunque nuestras funciones son directas, por dentro nos apoyamos en herramientas nativas de Haskell que sí son polimórficas. Por ejemplo, usamos `length`, `take` y `drop`, que pueden trabajar con listas de cualquier tipo (ya sean listas de números, letras o símbolos). También usamos `read` y `show`, que son súper flexibles y cambian de forma para poder traducir de texto a número y de número a texto.

4. Choose one of the predicates you developed and write down its declarative reading and its procedural reading. Explain in which situations both readings do not coincide and what consequences this has for the programmer

Predicado divisor

`divisor(DosCifras,Div):-`

`between(1, DosCifras, Div),`

`DosCifras mod Div == 0,`

`DosCifras \= Div.`

Lectura declarativa

"Un número `Div` es un divisor propio de un número de `DosCifras` si y solo si `Div` está comprendido entre 1 y el número de `DosCifras`, el número de `DosCifras` es divisible exactamente por `Div` (residuo cero), y `Div` es diferente de `DosCifras`."

Lectura Procedimental

1. Generar un número Div comenzando desde 1 hasta un numero de DosCifras (utilizando between/3)
2. Verificar si DosCifras dividido por Div arroja residuo 0
3. Verificar que Div sea distinto de DosCifras
4. Si alguna prueba falla, **retroceder (backtrack)** al generador between y probar con el siguiente Div
5. Repetir hasta agotar todos los valores de Div en el rango

En que situaciones no coinciden

Declarativamente, divisor (Div, 6) (buscar números que sean divisores de 6) debería funcionar igualmente que divisor (6, Div) (encontrar divisores de 6). Pero procedimentalmente:

?- divisor(6, X).

X = 1 ;

X = 2 ;

X = 3 . % Funciona: entre(1,6,...) genera valores

?- divisor(Y, 6). % Falla: Y queda sin valor en between(1, Y, 6) % No se puede usar un predicado generador "hacia atrás"

Consecuencias para el programador

1. No es suficiente pensar lógicamente: Aunque una relación se escriba correctamente, también debe ser correcta procedimentalmente.
2. El orden de las cláusulas y metas importa: En lógica pura, el orden no afecta la verdad. En Prolog, afecta:
 - Eficiencia: Cuántas soluciones se generan innecesariamente
 - Funcionalidad: Si un predicado está instanciado o no
 - Determinismo: Cuántas soluciones se producen
3. No todos los predicados funcionan de forma reversible. Hay que ser consciente de cuál dirección es la "natural" y cuál requiere cuidado especial.

4. El backtracking es transparente en la lectura declarativa: Una cláusula `divisor(DosCifras, Div) :- ...` no dice explícitamente que va a generar múltiples Div. Pero procedimentalmente, es lo que sucede. El programador debe entender esta brecha para predecir el comportamiento de su código.
5. Debugging requiere pensar procedimentalmente: Si `suma/2` no suma correctamente, no es suficiente verificar que la relación "es lógicamente correcta"; hay que rastrear cómo se ejecuta, entender el backtracking, y ver qué `findall` realmente recolecta.

5. Explain how unification and backtracking take part in the computation of the aliquot sum. State whether you used the cut (!) operator, at which points, and what effect its removal would have on the set of answers produced by the program.

`divisor(DosCifras,Div):-`

`between(1, DosCifras, Div),`

`DosCifras mod Div == 0,`

`DosCifras \= Div.`

`suma(DosCifras,SumAlicuata):-`

`findall(Div, divisor(DosCifras, Div), ListaDivisores),`

`sum_list(ListaDivisores,SumAlicuata).`

`categoria(DosCifras,SumAlicuata,'Engineering'):-`

`SumAlicuota == DosCifras.`

`categoria(DosCifras,SumAlicuata,'Administrative'):-`

`SumAlicuota > DosCifras.`

`categoria(DosCifras,SumAlicuata,'Humanities'):-`

`SumAlicuota < DosCifras.`

Participación de la unificación

En `suma(DosCifras, SumAlicuata)`, `findall/3` unifica `DosCifras` con el valor concreto (por ejemplo, 12) y luego unifica `Div` con cada divisor encontrado (1, 2, 3, 4, 6). Finalmente, `SumAlicuata` se unifica con el resultado de `sum_list` (16). Sin unificación, no habría forma de pasar valores ni recolectar resultados.

Participación del backtracking

`between(1, DosCifras, Div)` genera valores secuencialmente. Para cada valor, se prueba si `DosCifras mod Div =:= 0` y `DosCifras >= Div`. Si falla, el backtracking retrocede automáticamente al generador `between` y prueba el siguiente valor. `findall/3` explota este backtracking para recolectar todos los divisores. Sin backtracking, solo obtendríamos `Div = 1` y la suma alícuota sería incorrecta.

Uso del corte

No usé el operador `!` en mi solución. En `divisor/2` no es necesario porque queremos todas las soluciones. En `categoria/3` no es necesario porque las tres condiciones (`=:=`, `>`, `<`) son matemáticamente excluyentes: un número no puede ser simultáneamente igual, mayor y menor que otro. El corte sería redundante.

Efecto de eliminar backtracking

Si agregara `!` en `divisor/2`, `findall` solo recolectaría el primer divisor (1), `suma` daría un resultado incorrecto, y la categoría se clasificaría mal. Con `!` en `categoria/3`, el resultado sería el mismo porque las condiciones ya son excluyentes, pero el código sería menos limpio y más frágil si alguien cambiara el orden de las cláusulas.

6. Determine which of your predicates are reversible, that is, which admit more than one instantiation mode, and which are not. Justify in each School of Applied Sciences and Engineering Systems Engineering SI1002 Languages and Paradigms of Computation 2026-2 First Practice (15%) case what prevents reversibility, and include the query executed in "generate" mode together with the answers obtained.

Predicado reversible

caracteristicas_codigo/5

Es reversible porque su primera meta, `cifras/4`, usa `between(26201001, 29299999, Num)`, que genera valores de `Num` cuando está libre. Una vez generado `Num`, todos los demás argumentos (`PrimerasTresCifras`, `TresCifrasNum`, `DosCifras`) quedan instanciados automáticamente por aritmética, así que el resto de las metas del cuerpo se ejecutan en modo verificación sin problema.

Predicados no reversibles

Divisor/2

`between(1, DosCifras, Div)` exige que `DosCifras` esté instanciado; si se deja libre, `between` lanza error. Solo `Div` puede generarse, nunca `DosCifras`.

Suma/2

Depende de `divisor/2`, por lo que hereda la misma restricción: `DosCifras` debe estar dado.

Categoría/3

Las comparaciones `=`, `>`, `<` requieren que `DosCifras` y `SumAlicuata` estén instanciados como números; no se puede pedir "qué número tiene esta categoría" sin fijar ambos valores primero.

TraducirPeriodo/2

Aunque parece simétrico, cada cláusula usa `PrimerasTresCifras =:= 262`, una comparación aritmética. Si se llama con el primer argumento libre (por ejemplo, buscando qué dígitos corresponden a '2026-2'), la comparación falla porque `=:=` no puede evaluar una variable no instanciada. Solo funciona en el sentido dígitos → período.

Admision/2

`atom_number/2` y `atom_concat/3` son individualmente reversibles, pero en el cuerpo del predicado `atom_number(NumAtom, TresCifrasNum)` se ejecuta *antes* que `atom_concat`, así que si se llama con `TresCifrasNum` libre, ambos argumentos de `atom_number` están sin instanciar al mismo tiempo y falla. El orden de las metas impide la reversibilidad, aunque los predicados base sí la permitirían con otro orden.

Paridad/2

El operador mod requiere que Num esté instanciado; no puede generar números con una paridad dada.

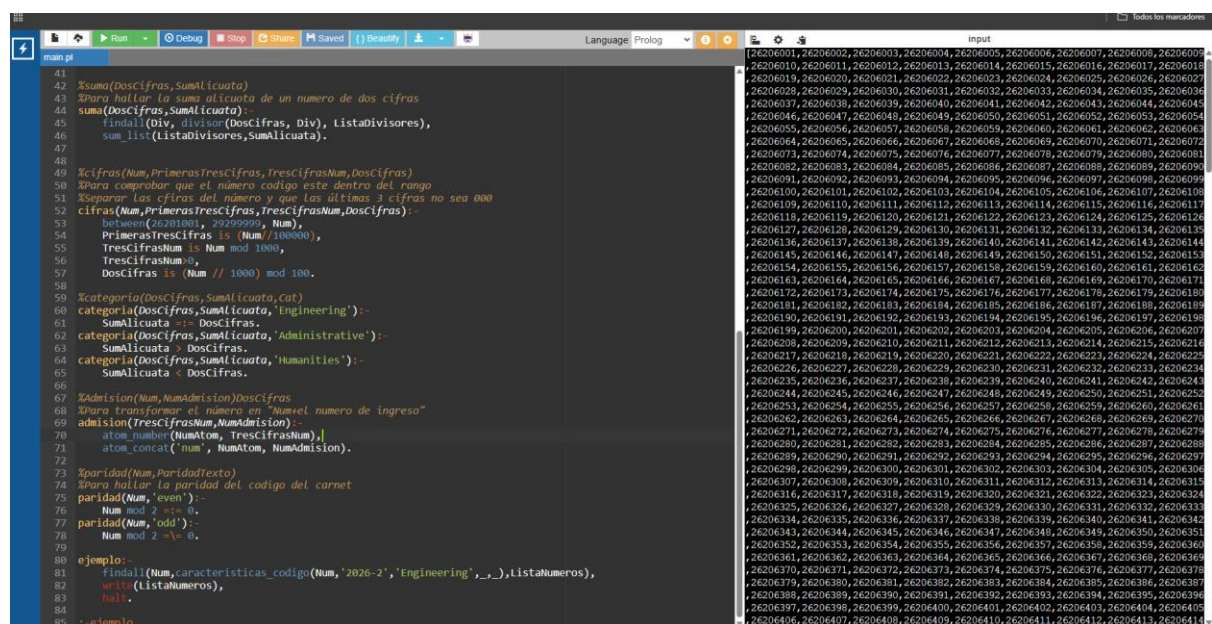
Evidencias de consultas:

Se utilizo la herramienta <https://swish.swi-prolog.org/> para hacer las consultas con "?-", pero en online GDB también se hizo, utilizando de findall y agregando la línea de código al final de dado que la consola de OnlineGDB no permite ese tipo de consultas ejemplo:-

```
findall(Num,caracteristicas_codigo(Num,'2026-2','Engineering',_,_),ListaNumeros),
write(ListaNumeros),

halt.
```

:-ejemplo.

The screenshot shows a Prolog IDE with a query on the left and a list of results on the right. The query is:

```
findall(Num,caracteristicas_codigo(Num,'2026-2','Engineering',_,_),ListaNumeros),
write(ListaNumeros),
halt.
```

 The results are a long list of numbers, starting with 26206001 and ending with 26206414. The IDE interface includes a menu bar, a toolbar, and a status bar.

De un mismo modo se podía hacer una consulta o verificación de un código con read y format agregando este código al final

traducirCodigo:-

```
write('Ingrese el codigo: '),
```

```
read(Num),
```

```
( caracteristicas_codigo(Num, Periodo, Cat, NumAdmision, ParidadTexto)
```

```
-> format('~w ~w ~w ~w~n', [Periodo, Cat, NumAdmision, ParidadTexto])
```

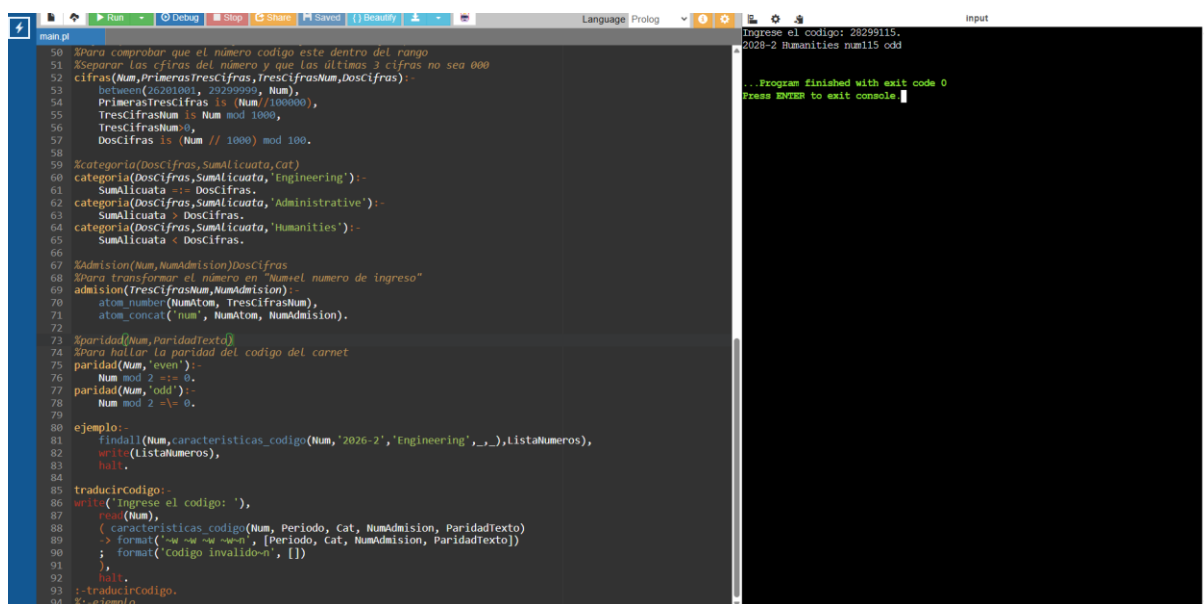
```
; format('Codigo invalido~n', [])
```

```
),
```

```
halt.
```

:-traducirCodigo.

Para fines de organización en el código fuente se puso como comentarios la ejecución automática



The screenshot shows a Prolog IDE with a source code editor on the left and a console window on the right. The source code is in Prolog and includes comments in Spanish. The console window shows the program's execution, including the prompt 'Ingrese el codigo: 28299115.' and the output '2028-2 Humanities num115 odd'. The program finished with exit code 0.

```
man.pl
50 %Para comprobar que el número código este dentro del rango
51 %Separar las cifras del número y que las últimas 3 cifras no sea 000
52 cifras(Num,PrimerasTresCifras,TresCifrasNum,DosCifras):-
53     between(26201001, 29299999, Num),
54     PrimerasTresCifras is (Num // 100000),
55     TresCifrasNum is Num mod 1000,
56     TresCifrasNum > 0,
57     DosCifras is (Num // 1000) mod 100.
58
59 %categoria(DosCifras,SumAlicuata,Cat)
60 categoria(DosCifras,SumAlicuata,'Engineering'):-
61     SumAlicuata =:= DosCifras.
62 categoria(DosCifras,SumAlicuata,'Administrative'):-
63     SumAlicuata > DosCifras.
64 categoria(DosCifras,SumAlicuata,'Humanities'):-
65     SumAlicuata < DosCifras.
66
67 %Admision(Num,NumAdmision)DosCifras
68 %Para transformar el número en "número de ingreso"
69 admision(TresCifrasNum,NumAdmision):-
70     atom_number(NumAtom,TresCifrasNum),
71     atom_concat('num', NumAtom, NumAdmision).
72
73 %paridad(Num,ParidadTexto)
74 %Para hallar la paridad del código del carnet
75 paridad(Num,'even'):-
76     Num mod 2 =:= 0.
77 paridad(Num,'odd'):-
78     Num mod 2 =\= 0.
79
80 ejemplo:-
81     findall(Num,caracteristicas_codigo(Num,'2026-2','Engineering',_,_),ListaNumeros),
82     write(ListaNumeros),
83     halt.
84
85 traducirCodigo:-
86     write('Ingrese el codigo: '),
87     read(Num),
88     ( caracteristicas_codigo(Num, Periodo, Cat, NumAdmision, ParidadTexto)
89     -> format('~w ~w ~w ~w~n', [Periodo, Cat, NumAdmision, ParidadTexto])
90     ; format('Codigo invalido~n', [])
91     ),
92     halt.
93 :-traducirCodigo.
94 %:-ejemplo.
```

Input: Ingrese el codigo: 28299115.
Output: 2028-2 Humanities num115 odd
... Program finished with exit code 0
Press ENTER to exit console

```
main.pl
50 %Para comprobar que el número código este dentro del rango
51 %Separar las cifras del número y que las últimas 3 cifras no sea 000
52 cifras(Num,PrimerasTresCifras,TresCifrasNum,DosCifras):-
53     between(26201001, 29299999, Num),
54     PrimerasTresCifras is (Num/100000),
55     TresCifrasNum is Num mod 1000,
56     TresCifrasNum > 0,
57     DosCifras is (Num // 1000) mod 100.
58
59 %categoria(DosCifras,SumAlicuata,Cat)
60 categoria(DosCifras,SumAlicuata,'Engineering'):-
61     SumAlicuata < DosCifras.
62 categoria(DosCifras,SumAlicuata,'Administrative'):-
63     SumAlicuata > DosCifras.
64 categoria(DosCifras,SumAlicuata,'Humanities'):-
65     SumAlicuata < DosCifras.
66
67 %Admision(Num,NumAdmision)DosCifras
68 %Para transformar el número en "Num=el número de ingreso"
69 admision(TresCifrasNum,NumAdmision):-
70     atom_number(NumAtom,TresCifrasNum),
71     atom_concat('num',NumAtom,NumAdmision).
72
73 %paridad(Num,ParidadTexto)
74 %Para hallar la paridad del código del carnet
75 paridad(Num,'even'):-
76     Num mod 2 =:= 0.
77 paridad(Num,'odd'):-
78     Num mod 2 =\= 0.
79
80 ejemplo:-
81     findall(Num,caracteristicas_codigo(Num,'2026-2','Engineering',_,_),ListaNumeros),
82     write(ListaNumeros),
83     halt.
84
85 traducirCodigo:-
86     write('Ingrese el código: '),
87     read(Num),
88     ( caracteristicas_codigo(Num,Periodo,Cat,NumAdmision,ParidadTexto)
89     -> format('~w ~w ~w ~w ~w\n',[Periodo,Cat,NumAdmision,ParidadTexto])
90     ; format('Código inválido\n',[ ]))
91     ),
92     halt.
93 :-traducirCodigo.
94 %- ejemplo.
```

```
Ingrese el código: 1.
Código inválido
...Program finished with exit code 0
Press ENTER to exit console.
```

SWISH

File Edit Examples Help

Program

```
38 between(1, Descifras, Div),
39 Descifras mod Div <= 0,
40 Descifras <= Div.
41
42 suma(Descifras, SumAlcuota)
43 Para hallar la suma alcuota de un numero de dos cifras
44 suma(Descifras, SumAlcuota):-
45     findall(Div, divisor(Descifras, Div), ListaDivisores),
46     sum_list(ListaDivisores, SumAlcuota).
47
48
49 Xcifras(Num, PrimerasTresCifras, TresCifrasNum, Descifras)
50 Para comprobar que el número código este dentro del rango
51 Xcifras(Num, PrimerasTresCifras, TresCifrasNum, Descifras):-
52     cifras(Num, PrimerasTresCifras, TresCifrasNum, Descifras):-
53         between(26201001, 29299999, Num),
54         PrimerasTresCifras is (Num//100000),
55         TresCifrasNum is Num mod 1000,
56         TresCifrasNum > 0,
57         Descifras is (Num // 1000) mod 100.
58
59 Xcategoria(Descifras, SumAlcuota, Cat)
60 categoria(Descifras, SumAlcuota, 'Engineering'):-
61     SumAlcuota >= Descifras.
62 categoria(Descifras, SumAlcuota, 'Administrative'):-
63     SumAlcuota > Descifras.
64 categoria(Descifras, SumAlcuota, 'Humanities'):-
65     SumAlcuota < Descifras.
66
67 Xadmission(Num, NumAdmission, Descifras)
68 Para transformar el número en "Num" el número de ingreso
69 admission(TresCifrasNum, NumAdmission):-
70     atom_number(NumAdmission, TresCifrasNum),
71     atom_concat('num', NumAdmission, NumAdmission).
72
73 Xporidad(Num, PoridadTexto)
74 Para hallar la poridad del código del carnet
75 poridad(Num, 'even'):-
76     Num mod 2 <= 0.
77 poridad(Num, 'odd'):-
78     Num mod 2 > 0.
79
80
```

caracteristicas_codigo(26299115, P, C, A, Pa), format('~w ~w ~w ~w', [P, C, A, Pa]).

2028-2 Humanities num115 odd
A = num115.
C = 'Humanities'.
P = '2028-2'.
Pa = odd.
Next 10 100 1.000 Stop

caracteristicas_codigo(28124236, P, C, A, Pa), format('~w ~w ~w ~w', [P, C, A, Pa]).

2028-1 Administrative num236 even
A = num236.
C = 'Administrative'.
P = '2028-1'.
Pa = even.
Next 10 100 1.000 Stop

caracteristicas_codigo(28124236, P, C, A, Pa), format('~w ~w ~w ~w', [P, C, A, Pa]).

2028-1 Administrative num236 even
A = num236.
C = 'Administrative'.
P = '2028-1'.
Pa = even.

caracteristicas_codigo(27128112, P, C, A, Pa), format('~w ~w ~w ~w', [P, C, A, Pa]).

2027-1 Engineering num112 even
A = num112.
C = 'Engineering'.
P = '2027-1'.
Pa = even.

caracteristicas_codigo(26276002, P, C, A, Pa), format('~w ~w ~w ~w', [P, C, A, Pa]).

2026-2 Humanities num2 even
A = num2.
C = 'Humanities'.
P = '2026-2'.
Pa = even.
Next 10 100 1.000 Stop

?- caracteristicas_codigo(26276002, P, C, A, Pa), format('~w ~w ~w ~w', [P, C, A, Pa]).

Examples History Solutions

table results Run

caracteristicas_codigo(26276002, _, _, _, _)

false

caracteristicas_codigo(26276002, _, _, _, _)

true

caracteristicas_codigo(-26276002, _, _, _, _)

false

caracteristicas_codigo(Num, _, _, _, _)

Num = 26201001
Num = 26201002
Num = 26201003
Num = 26201004
Num = 26201005
Num = 26201006
Num = 26201007
Num = 26201008
Num = 26201009
Num = 26201010
Num = 26201011
Num = 26201012
Num = 26201013
Num = 26201014
Num = 26201015
Num = 26201016
Num = 26201017
Num = 26201018
Num = 26201019
Num = 26201020
Num = 26201021
Num = 26201022
Num = 26201023
Num = 26201024
Num = 26201025
Num = 26201026
Num = 26201027
Num = 26201028
Num = 26201029
Num = 26201030

?- caracteristicas_codigo(Num, _, _, _, _)

Examples History Solutions

table results Run

 `caracteristicas_codigo(Num,Humanities, _, _, _)`

Humanities = '2026-2',
Num = 26201001
Humanities = '2026-2',
Num = 26201002
Humanities = '2026-2',
Num = 26201003
Humanities = '2026-2',
Num = 26201004
Humanities = '2026-2',
Num = 26201005
Humanities = '2026-2',
Num = 26201006
Humanities = '2026-2',
Num = 26201007
Humanities = '2026-2',
Num = 26201008
Humanities = '2026-2',
Num = 26201009
Humanities = '2026-2',
Num = 26201010
Humanities = '2026-2',
Num = 26201011
Humanities = '2026-2',
Num = 26201012
Humanities = '2026-2',
Num = 26201013
Humanities = '2026-2',
Num = 26201014
Humanities = '2026-2',
Num = 26201015
Humanities = '2026-2',
Num = 26201016
Humanities = '2026-2',
Num = 26201017
Humanities = '2026-2',
Num = 26201018
Humanities = '2026-2',
Num = 26201019
Humanities = '2026-2'

?

`caracteristicas_codigo(Num,Humanities, _, _, _)`

Examples▲History▲Solutions▲

☐ table results Run!

```
caracteristicas_codigo(Num,Humanities,_,num999,_)

Humanities = '2026-2',
Num = 26201999
Humanities = '2026-2',
Num = 26202999
Humanities = '2026-2',
Num = 26203999
Humanities = '2026-2',
Num = 26204999
Humanities = '2026-2',
Num = 26205999
Humanities = '2026-2',
Num = 26206999
Humanities = '2026-2',
Num = 26207999
Humanities = '2026-2',
Num = 26208999
Humanities = '2026-2',
Num = 26209999
Humanities = '2026-2',
Num = 26210999
Humanities = '2026-2',
Num = 26211999
Humanities = '2026-2',
Num = 26212999
Humanities = '2026-2',
Num = 26213999
Humanities = '2026-2',
Num = 26214999
Humanities = '2026-2',
Num = 26215999
Humanities = '2026-2',
Num = 26216999
Humanities = '2026-2',
Num = 26217999
Humanities = '2026-2'

?- caracteristicas_codigo(Num,Humanities,_,num999,_)

```

7. Compare both solutions and indicate which parts of the problem were expressed more naturally in the functional paradigm and which in the logic paradigm. Identify at least one aspect of the solution in which the degree of declarativity is greater in one paradigm than in the other, and justify your answer.

Partes naturales en el paradigma funcional

La extracción y transformación de datos del código se expresa de forma muy directa con funciones puras y listas por comprensión. `divisoresPropios` es prácticamente una traducción literal de la definición matemática: $[x \mid x < [1..n], n \bmod x \neq 0]$. Igualmente, `procesarCodigo` usa guardas (`|`) y `where` para descomponer el string en partes (periodo, categoría, consecutivo) de manera lineal y legible, como una receta

paso a paso. La validación de formato (longitud 8, periodo válido) también es natural en Haskell porque son chequeos secuenciales sobre un valor ya dado.

Partes más naturales en el paradigma lógico

La generación de códigos ("modo generar") es donde Prolog brilla y Haskell no tiene equivalente directo: `caracteristicas_codigo(Num, '2029-2', 'Engineering', _, _)` con Num libre permite que `between/3` produzca automáticamente todos los códigos que cumplen una condición, usando el mismo predicado que sirve para verificar. En Haskell habría que escribir una función distinta y explícita (`filter`, recursión) para lograr lo mismo. La clasificación de Nicómaco (`categoria/3`) también resulta natural en Prolog porque se expresa como tres relaciones independientes y mutuamente excluyentes por su propia naturaleza aritmética, sin necesidad de una estructura de control como `if/guards` que impone un orden de evaluación.

Aspecto con mayor grado de declaratividad

En Prolog, `caracteristicas_codigo/5` es una **relación**, no una función: no dice "cómo calcular la salida a partir de la entrada", dice "qué debe cumplirse entre Num y sus 4 características". Por eso el mismo predicado sirve para verificar, describir y generar sin cambiar una sola línea. Esto es más declarativo que la solución en Haskell, donde `procesarCodigo :: String -> String` es intrínsecamente direccional: solo puede ir de código a descripción, nunca al revés, porque una función matemática por definición mapea en una sola dirección.

La declaratividad se mide por cuánto describe el código *qué* es verdadero en lugar de *como* computarlo. Haskell describe una transformación (entrada → salida) mediante evaluación perezosa y guardas, lo cual sigue siendo un cálculo dirigido. Prolog describe una relación lógica que el motor de inferencia puede recorrer en cualquier dirección donde haya suficiente instanciación (gracias a `between/3` como generador), lo que representa un nivel de declaratividad estrictamente mayor: el programador no programó "cómo generar códigos", solo definió qué hace válido a un código, y la generación surgió como consecuencia gratuita de esa definición.

Link Video

<https://youtu.be/bpePFp3Kxnc>